La documentazione di rsedit

Regole, inventario e piano del riordino — passo 0

Indice

1 In breve 2
2 Le regole 3
2.1 Chi legge che cosa 3
2.2 Dove sta ogni informazione 3
2.3 I manpage 4
2.4 Le docstring 5
2.5 Il rustdoc 6
2.6 I commenti nel codice 6
2.7 Il manuale per sviluppatori 6
2.8 Il rapporto sui problemi 6
2.9 La storia e i commit 7
2.10 I test 7
2.11 Le copie controllate 7
2.12 La lista di controllo di ogni patch 7
2.13 Che cosa si fa dei commenti di oggi 8
2.14 Tre esempi 8
3 L'inventario 11
3.1 Quanto c'è 11
3.2 I commenti del Rust 11
3.3 Docstring e rustdoc 12
3.4 I moduli Lisp 12
3.5 I manpage 13
3.6 I documenti in `doc/` e nella radice 13
3.7 Lo stesso fatto in più posti 14
3.8 Nomi citati che non esistono 15
3.9 Difetti trovati durante l'inventario 15
4 Il piano 16
4.1 Fase A — le regole e i controlli (passo 1, fatto) 16
4.2 Fase B — i manpage e le docstring (passi 2–9) 16
4.3 Fase C — il codice e i commenti (passi 10–21) 17
4.4 Fase D — il manuale (passo 22) 18
4.5 Come si consegna ogni passo 18
5 Le decisioni 19

1 In breve

Oggi la stessa informazione su rsedit è scritta in più posti: nei commenti del codice, nelle docstring che `C-h f` mostra, nei manpage di `man/` e nei documenti typst di `doc/`. Le copie non si aggiornano insieme e divergono. L'elenco dei moduli caricati all'avvio, per esempio, compare in sei posti, in cinque versioni diverse (§ 3.7). Nei commenti ci sono 145 sezioni `# Why` e circa 80 righe che raccontano com'era il codice prima. Il manuale per sviluppatori descrive ancora lo scheduler che le patch 1–9 hanno sostituito.

Questo documento stabilisce dove sta ogni informazione, come si scrive e come si controlla che non diverga. Poi misura lo stato attuale e propone i passi per arrivarci.

> **Stato: approvato; il passo 1 è fatto.** Le misure dell'inventario si riferiscono alla revisione `58d0b0d`. Le decisioni prese sono raccolte nel § 5.

> **Le regole, in sei righe.**
> 1. **Ogni fatto ha una casa sola.** Altrove si rimanda, non si copia.
> 2. **Chi usa l'editor legge i manpage**, completi e con esempi pratici. La docstring è il promemoria breve che `C-h f` mostra, e porta alla pagina.
> 3. **Chi sviluppa legge il manuale typst**: architettura, thread, lock e decisioni, con diagrammi. Il rustdoc è il contratto breve di ogni elemento.
> 4. **Un commento dice solo ciò che il codice non può dire**: un'invariante, un ordine di lock, un perché non ovvio, un TODO.
> 5. **Nel codice c'è solo il presente.** La storia sta nei commit, i difetti nel rapporto sui problemi.
> 6. **Una copia è ammessa solo se un test la confronta con la sua fonte.**

Il § 2 è la parte permanente: vale per ogni modifica futura. Il § 3 fotografa lo stato di partenza e resta, come termine di confronto, fino all'ultimo passo. Il § 4 propone i passi del riordino e si accorcia man mano che vengono fatti.

2 Le regole

2.1 Chi legge che cosa

Chi	Dove guarda	Che cosa cerca
Chi usa l'editor	i manpage: `C-h m`, `M-x manpage`	come si fa una cosa, con esempi da copiare; i tasti; i limiti
Chi configura l'editor o scrive moduli Lisp	i manpage e le docstring: `C-h f`, `C-h v`	in più, la firma esatta di una funzione e il nome di una variabile
Chi sviluppa il Rust	il manuale in `doc/`, il rustdoc, i commenti	come è fatto il codice, perché, dove intervenire
Chi indaga un difetto	`doc/rapporto-problemi.typ` `git log`	che cosa è già noto, quando un comportamento è cambiato

2.2 Dove sta ogni informazione

Informazione	Casa	Altrove
Che cosa fa una funzione, un comando o una variabile: firma, argomenti, valore	la sua docstring	la SYNOPSIS della sua pagina la elenca; `C-h f` aggiunge il rimando alla pagina
Come si usa una funzionalità: concetti, combinazioni, esempi, limiti	il manpage del suo argomento	nessuna copia nelle docstring
I tasti	i keymap dei moduli Lisp	le sezioni KEYS delle pagine, controllate da un test; `C-h b` li calcola
I moduli caricati e il loro ordine	`DEFAULT_INIT_LISP` in `editor/boot.rs`	la sezione MODULES di `rsedit.txt`, controllata; `build.rs` e i test non ne tengono una propria
L'elenco delle pagine	la directory `man/`	l'indice di `guide.txt`, controllato; i test leggono la directory
Dove l'editor tiene i suoi file: configurazione, cache, log	il codice, `paths.rs`	la pagina `configuration`, per l'utente
La struttura: moduli, tipi principali, thread, lock, flussi	il manuale per sviluppatori, `doc/architettura-editor.typ`	la documentazione di modulo: poche righe e il rimando
Perché il codice è fatto così	le decisioni del capitolo del manuale	nel codice, al più una riga e il rimando
Invarianti, ordine dei lock, `SAFETY`, il thread su cui gira una funzione	il commento o il rustdoc accanto al codice	il manuale riassume l'ordine dei lock dell'intero editor
Il contratto di un elemento Rust: che cosa fa, errori, panic	il suo rustdoc	—
La storia: che cosa c'era prima e perché è cambiato	il messaggio di commit	ciò che ne resta valido è una decisione del manuale

Informazione	Casa	Altrove
I difetti noti e il lavoro rimandato	una scheda di `rapporto-problemi.typ`	`TODO(ID)` nel codice, finché la scheda è aperta
Il comportamento atteso, caso per caso	i test: il nome dice il comportamento	—
Come si misurano le prestazioni	`PERFORMANCE.md`	—
Che cos'è rsedit, come si compila, quali documenti ci sono e per chi	`README.md`	per tutto il resto rimanda ai manpage e ai documenti

`todo.org` è la lista personale dell'autore e resta fuori da queste regole.

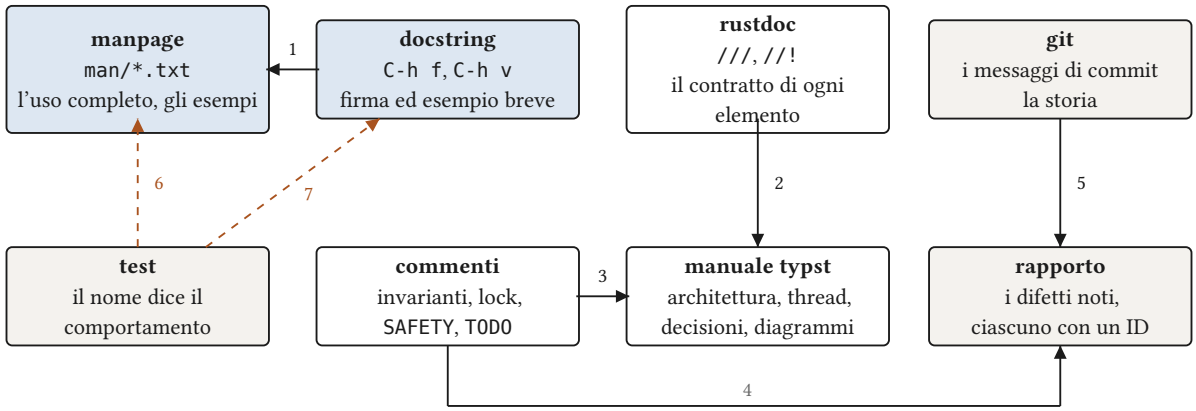


Figura 1: Le fonti e i loro rimandi. Frecce piene: «rimanda a»; tratteggiate: «controlla». In azzurro ciò che legge l'utente. (1) `C-h f` aggiunge «Manual:» con la pagina che elenca il nome nella sua SYNOPSIS. (2) La documentazione di modulo rimanda al capitolo, con un'etichetta. (3) Un perché lungo diventa una decisione del manuale. (4) `TODO(ID)` rimanda alla scheda. (5) Il commit che corregge un difetto chiude la sua scheda. (6, 7) I test confrontano le copie con le fonti e i nomi citati con quelli definiti.

2.3 I manpage

Una pagina per argomento, come oggi. Le pagine sono il riferimento completo per chi usa l'editor: ogni nome pubblico — funzione, comando, variabile, modo — ha una pagina-casa che lo elenca nella SYNOPSIS e ne spiega l'uso. Una docstring non contiene niente che manchi alla pagina.

- **SYNOPSIS** elenca i nomi di cui la pagina è la casa: `(nome ARGOMENTI)` per funzioni e comandi, il nome da solo per le variabili. Ogni nome pubblico compare nella SYNOPSIS di una sola pagina. Da qui `C-h f` e `C-h v` ricavano il rimando «Manual:», che quindi non si scrive a mano.
- **EXAMPLES** c'è in ogni pagina: esempi completi, che si possono incollare in `init.lisp` o in `M-:`, ciascuno preceduto da una frase che dice a che cosa serve.
- **KEYS** e **KEYS IN … MODE** restano come oggi: le controlla `guide_tests`.
- Le sezioni intermedie seguono ciò che il lettore vuole fare, non la struttura del codice. Il funzionamento interno va nel manuale per sviluppatori; la pagina dice ciò che l'utente vede e può fare, limiti compresi.
- In inglese, righe entro 78 colonne, intestazioni in maiuscolo a colonna 0 e corpo rientrato di quattro spazi: è ciò che `manpage-mode` riconosce e colora.

Il modello:

```
NAME
    dired - a directory in a buffer

SYNOPSIS
    (dired DIRECTORY)
    (dired-find-recursive)
    dired-sort
    find-file-recursive-limit

DESCRIPTION
    What it is for, and the idea behind it, in a few paragraphs.

OPENING A DIRECTORY
    One heading for each thing the reader wants to do.

EXAMPLES
    A sentence saying what the example is for, then the example,
    complete and ready to paste into init.lisp or M-:.

        (setq dired-sort 'type)        ; directories first

KEYS
    C-x d                   dired

KEYS IN DIRED MODE
    <ret>                   dired-find-file

SEE ALSO
    files, buffers
```

2.4 Le docstring

Valgono per le primitive (le costanti `*_DOC` di `primitives/`) e per le funzioni, i comandi e le variabili dei moduli Lisp. Seguono le convenzioni di Emacs, come già oggi.

```
(name ARGS): One sentence, in the imperative, saying what it does.

At most one short paragraph: what the arguments are, what comes back,
what is refused.

Example:
  (name ...)
```

- La prima riga basta da sola: è ciò che mostrano gli elenchi e i completamenti. Per le primitive comincia con la chiamata, `(name ARGS):`, come `doc_faithfulness_tests` già controlla.
- Gli argomenti in maiuscolo; gli altri nomi citati come `` `name' ``, la convenzione di Emacs: è così che il test dei nomi citati li riconosce.
- Un esempio quando l'uso non è ovvio.
- Di norma non più di 600 caratteri; un test rifiuta quelle oltre 1 000.
- Niente architettura, storia o motivazioni lunghe: vanno nella pagina o nel manuale. Alla pagina porta il rimando che `C-h f` aggiunge da sé.
- Le funzioni private, con `--` nel nome, hanno una riga che dice che cosa restituiscono. Un ragionamento interno è un commento `;;`, non una docstring.

2.5 Il rustdoc

Segue le Rust API Guidelines e le convenzioni della RFC 1574.

- **La riga di riassunto** è una frase alla terza persona («Returns …», «Creates …») che basta da sola: è ciò che `cargo doc` mostra negli elenchi.
- Poi, se serve, un paragrafo breve. Per gli elementi dell'interfaccia pubblica (i `pub use` di `lib.rs`): le sezioni `# Errors`, `# Panics` e `# Safety` quando si applicano, e `# Examples`.
- I tipi e le funzioni citati sono link (`` [`Log::flush`] ``) e devono risolversi: `cargo doc --document-private-items` senza avvisi. Oggi ce ne sono 18.
- Campi e varianti: una riga quando il nome non basta, niente quando basta.
- **La documentazione di modulo** (`//!`) dice in una-cinque righe di che cosa il modulo è responsabile, e rimanda al capitolo del manuale con la sua etichetta: `Design: doc/architettura-editor.typ, <sfondo>.` Un test controlla che l'etichetta esista.
- In inglese, come oggi.

2.6 I commenti nel codice

Restano	Escono
• un'invariante su cui il codice conta e che i tipi non esprimono; • l'ordine in cui si prendono i lock, il thread su cui una funzione gira; • `SAFETY:` prima di ogni `unsafe`; • un perché non ovvio, in una o due righe, con il rimando alla decisione se ne serve di più; • il motivo di un aggiramento: un terminale, un sistema operativo; • `TODO(ID): …`, dove ID è una scheda del rapporto o un'etichetta come quelle già in uso (`uncertain`, `improve`).	• ciò che il codice dice già; • la storia: «used to», «was … until», «the bug this fixes»; • le motivazioni lunghe, le sezioni `# Why`: diventano decisioni del manuale; • le spiegazioni per l'utente: vanno nel manpage; • i riferimenti a cose che non ci sono più; • gli ID di problemi già risolti.

Nei moduli Lisp l'intestazione `;;;` dice in una-cinque righe che cosa il modulo aggiunge, e rimanda alla sua pagina e al capitolo del manuale.

2.7 Il manuale per sviluppatori

- Un solo manuale, `doc/architettura-editor.typ`, con un capitolo per sottosistema. Ogni capitolo dice di che cosa il sottosistema è responsabile e ne descrive le strutture dati principali, i thread e i lock, i flussi (con diagrammi), come si estende (le ricette) e le decisioni.
- Una decisione è una voce breve: il problema, la scelta, che cosa comporta, l'alternativa scartata e perché. Prende il posto delle sezioni `# Why` del codice.
- I diagrammi usano gli aiuti di `style.typ`. Sono obbligatori almeno per gli strati e le dipendenze dei moduli, i thread, l'ordine dei lock, il percorso da un tasto a un comando, il lavoro asincrono e la fotografia del frame.
- Il codice si cita per nome di funzione o di tipo, non per numero di riga, che invecchia a ogni modifica. Gli estratti sono brevi.
- Le etichette (`<sfondo>`) non cambiano, perché il codice vi rimanda.
- In italiano, come oggi. Il PDF si rigenera nello stesso commit della sorgente.

2.8 Il rapporto sui problemi

- Una scheda per difetto, con un ID stabile, come oggi.

- Un campo nuovo, `stato`: `aperto` oppure `risolto`, con il commit che lo risolve. Le schede risolte escono dal riepilogo e passano in un'appendice compatta.
- Il codice cita un ID solo in un `TODO(ID)`, e solo finché la scheda è aperta.

2.9 La storia e i commit

Un commento che racconta la storia si cancella. Prima si guarda se contiene una decisione ancora valida: se sì, la decisione va nel manuale. Il resto della storia sta nel messaggio di commit, che dice che cosa cambia, perché, che cosa c'era prima e quale scheda chiude.

> Per questo le patch si applicano con `git am`, che conserva il loro messaggio di commit (decisione 7 del § 5).

2.10 I test

- Il nome di un test descrive il comportamento, come già si fa: `a_variable_set_but_empty_counts_as_not_set`.
- Un commento in un test spiega in una o due righe un caso non ovvio. La storia del difetto che il test previene va nel commit.
- I test di documentazione sono quelli che controllano le copie (§ 2.11).

2.11 Le copie controllate

Copia	Fonte	Test
le sezioni KEYS	i keymap	`guide_tests` (esiste)
i SEE ALSO	la directory `man/` e le funzioni	`guide_tests` (esiste)
l'indice di `guide.txt`	la directory `man/`	`guide_tests` (esiste)
la prima riga delle docstring delle primitive	il nome della primitiva	`doc_faithfulness_tests` (esiste)
la sezione MODULES di `rsedit.txt`	`DEFAULT_INIT_LISP`	`guide_tests`
le SYNOPSIS	i nomi definiti	`guide_tests`
i nomi citati in pagine e docstring	i nomi definiti	`doc_faithfulness_tests`
i rimandi dal codice al manuale	le etichette typst	`doc_pointer_tests`

I test partono con l'elenco delle eccezioni di oggi — un nome senza pagina, una docstring troppo lunga — che può solo accorciarsi. Gli elenchi stanno in `core/src/tests/doc_exceptions/`, raggruppati per il passo che li smaltisce, e un nome che vi resta dopo essere stato sistemato fa fallire il test. Così nessun passo deve correggere tutto subito, e nessuno può peggiorare le cose.

2.12 La lista di controllo di ogni patch

> - Comportamento visibile cambiato ⟶ la pagina, con i suoi esempi.
> - Nome pubblico nuovo ⟶ la docstring e la SYNOPSIS della sua pagina.
> - Struttura, thread o lock cambiati ⟶ il capitolo del manuale e il suo diagramma.
> - Una scelta non ovvia ⟶ una voce tra le decisioni.
> - Difetto corretto ⟶ la scheda passa a `risolto` e il `TODO(ID)` si toglie.

- Nessun commento storico, nessun rimando a cose tolte.
- Test verdi, compresi quelli di documentazione; `cargo doc` senza avvisi; i PDF rigenerati.

2.13 Che cosa si fa dei commenti di oggi

Ogni commento e ogni docstring esistente rientra in uno di questi casi. La tabella è lo strumento dei passi delle fasi B e C.

Il commento…	Che cosa se ne fa	Esempio, revisione `58d0b0d`
ripete il codice	si cancella	—
racconta la storia	si cancella; una decisione ancora valida va nel manuale	`managers/log.rs`: « `log_diagnostic` used to take the list's lock and, still holding it, `write_all` to the file…»
cita un problema risolto	si toglie l'ID, la regola resta	`Log::flush`: «A write that fails is dropped, never a panic (AVV-3)»
spiega all'utente come usare qualcosa	va nel manpage	`DEFAULT_INIT_LISP` rimanda a `common-keymaps.lisp` per spiegare `auto-save` e `watch-files`; la docstring di `background-call` (§ 2.14)
motiva una scelta di progetto	va tra le decisioni del manuale; sul posto, se serve, una riga e il rimando	`paths.rs`: «Why macOS gets `~/.config` and `~/.cache`»
enuncia un'invariante, un ordine di lock, un thread	resta, in una-tre righe	`Log::flush`: «The file's lock is taken before the lines are…»
descrive lavoro da fare	una scheda del rapporto e `TODO(ID)`	`paths.rs`: il TODO sulla scrivibilità della directory
è falso, o cita cose che non esistono	si corregge o si cancella	`lib.rs` elenca un modulo `lisp/`, che è nel crate `risp`; cinque rimandi a `rsedit-man` (§ 3.8)

2.14 Tre esempi

2.14.1 Una docstring: `background-call`

Oggi la docstring è lunga 3 689 caratteri. Eccone l'inizio:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Do JOB in
the background under the name NAME, and return NAME at once without waiting
for it.

This is how anything that may take longer than a frame is started. JOB is
either a function of no arguments -- called once, in the background -- or a
`fiber', which is resumed a step at a time and is the shape to use when the
work can be done in pieces:
  [un esempio di nove righe e altri otto paragrafi]
```

Quasi tutto ripete la pagina `background`, che ha lo stesso esempio e le stesse regole. Dopo il riordino:

```
(background-call NAME JOB &optional ON-PROGRESS ON-DONE INTERVAL): Run JOB in
the background under NAME, and return NAME at once.

JOB is a function of no arguments, called once, or a `fiber', resumed a step
at a time. ON-PROGRESS is called with NAME after each step but the last, and
ON-DONE with NAME and t or nil when JOB ends; both run between commands. A
job started under a name in use replaces the one running. INTERVAL is how
many milliseconds a plain (yield) waits: 40 by default.

Example:
  (defun my-done (name ok) (message "%s finished: %s" name ok))
  (background-call 'my-sum (lambda () (setq my-sum (my-slow-sum))) nil 'my-done)

Manual: background
```

L'ultima riga non è scritta nella docstring: la aggiunge `C-h f`, dalla SYNOPSIS della pagina. La formulazione definitiva si scrive nel passo 8.

2.14.2 Un rustdoc: `app_dir`

Oggi:

```
/// The directory OPTIONS asks for, where this system keeps it -- made first,
/// when it is missing and OPTIONS says to. See the module documentation for
/// where that is.
```

Dopo, con le sezioni della RFC 1574:

```
/// Returns the directory `options.dir` names on this system, creating it and
/// its parents first when it is missing and `options.create` is set.
///
/// Where each directory is: the `configuration` manual page.
///
/// # Errors
///
/// [`DirError::Undetermined`] when nothing in the environment says where the
/// directory is; [`DirError::NotFound`] when it is missing and `create` is
/// false; [`DirError::NotADirectory`] or [`DirError::CannotCreate`] otherwise.
///
/// # Examples
///
/// ```no_run
/// use rsedit_core::{AppDir, DirOptions, app_dir};
///
/// let cache = app_dir(DirOptions { dir: AppDir::Cache, create: true })?;
/// # Ok::<(), rsedit_core::DirError>(())
/// ```
```

La tabella delle precedenze, oggi nella documentazione del modulo, è un'informazione per l'utente: passa nella pagina `configuration`, che ne ha già una parte. Le due sezioni `# Why`, sul macOS e sul Windows, diventano decisioni del capitolo sui file dell'editor.

2.14.3 Una documentazione di modulo: `managers/log.rs`

Oggi sono 29 righe:

```
//! Diagnostics: what has been logged, and where a copy of it goes.
//!
//! # Why these two are one thing
//!   [7 righe: la lista e il file si aggiornano insieme, e perché]
//! # Who writes the file
//!   [10 righe: append non scrive, flush sì, e lo chiama il watchdog]
//!
//! `log_diagnostic` used to take the list's lock and, still holding it,
//! `write_all` to the file, which put a disk write inside a lock every
//! diagnostic in the editor had to queue behind. The flush writes with the
//! list's lock given back.
```

Dopo:

```
//! Diagnostics: the list of what has been logged, and the file it is
//! copied to. [`Log::append`] only records; [`Log::flush`], which the
//! WorkManager's watchdog calls, writes what is owed.
//!
//! Design: doc/architettura-editor.typ, <log>.
```

Il primo `# Why` diventa la decisione «La lista e il file si aggiornano insieme». Chi scrive il file, e su quale thread, va nel capitolo sui thread. La storia si cancella (§ 2.9).

3 L’inventario

Misure sulla revisione `58d0b0d`, ottenute con script che contano le righe e cercano i nomi; sono fotografie, non giudizi.

3.1 Quanto c’è

Parte	codice	documentazione	commenti	quota
Rust, `core/src` e `src`, senza i test	23 832	8 433	2 699	32 %
Rust, test	34 439	2 699	3 020	14 %
Lisp, `core/lisp` (22 moduli)	3 033	1 004	1 632	46 %
manpage, `man/` (22 pagine)	—	2 225	—	—
typst, `doc/` (5 sorgenti)	—	5 865	—	—

Righe. Nel Rust «documentazione» sono le righe `///` e `//!`, «commenti» le righe `//`; nel Lisp «documentazione» sono le docstring e «commenti» le righe `;`. La quota è documentazione più commenti sul totale. Le docstring delle primitive sono stringhe Rust e qui contano come codice: 2 491 righe.

3.2 I commenti del Rust

Modulo	codice	documentazione	commenti	quota
`primitives/`	11 628	1 604	1 030	18 %
`editor/`	2 423	1 527	519	46 %
`work/`	1 996	667	72	27 %
`buffer/`	1 533	885	175	41 %
`modes/`	1 403	682	186	38 %
`managers/`	1 366	1 133	212	50 %
`ui/`	1 348	724	187	40 %
`text/`	755	491	64	42 %
`src/` (la TUI)	629	134	170	33 %
`input.rs`	241	120	8	35 %
`feature/`	211	201	56	55 %
`commands.rs`	136	104	3	44 %
`paths.rs`	132	102	7	45 %
`lib.rs`	31	59	10	69 %

La quota bassa di `primitives/` è un effetto del conteggio: le sue docstring contano come codice.

Le righe di commento fuori dai test sono 11 132. Di queste, 3 269 stanno in 128 blocchi di almeno 15 righe consecutive, e 17 blocchi superano le 40. Alcune formule segnalano i commenti da rivedere:

Formula	fuori dai test	nei test
storia: «used to», «no longer», «the bug this…»	78	57
sezioni `# Why`	145	16
«for now», «today», «not yet»	38	5
ID del rapporto (`AVV-3`, …)	2	5

Formula	fuori dai test	nei test
`TODO`	4	0

Le righe storiche sono per lo più storia vera («It used to keep the renderer waking every 40 ms»), con qualche falso positivo («text that no longer exists»). I quattro `TODO` sono in `editor/log.rs`, `paths.rs`, `primitives/ui.rs` e `src/main.rs`; nessuno rimanda a una scheda.

3.3 Docstring e rustdoc

Misura	Valore
docstring delle primitive	296 costanti, 2 491 righe di sorgente
lunghezza: mediana, 90° percentile, massimo	469, 1 089, 3 689 caratteri (`background-call`)
oltre 1 500 caratteri	8: `background-call`, `set-transient-keymap`, `define-worker`, `shell-command-start`, `directory-files-recursive` e altre tre
con un esempio; con un rimando a una pagina	230; 2
docstring dei moduli Lisp	251; mediana 161 caratteri, massimo 1 656 — una funzione privata, `electric-pair--closed-ahead-p`
definizioni Lisp private (`--`)	212 su 319
blocchi `///`	1 564: 919 di una-tre righe, 530 di quattro-dieci, 98 di 11–20, 17 oltre le 20
documentazione di modulo `//!`	103 file: 16 senza, 16 di una-cinque righe, 20 di 6–20, 50 di 21–60, 1 oltre (`primitives/lisp_job.rs`, 78)
avvisi di `cargo doc --document-private-items`	18, quasi tutti link a moduli privati

3.4 I moduli Lisp

Modulo	righe	codice	commenti	docstring	intestazione
`vim`	1 067	1 006	19	24	5
`dired`	618	221	174	142	51
`help`	565	331	72	88	34
`completion`	512	162	158	123	45
`manpage`	485	162	119	134	32
`indent`	410	161	106	85	32
`common-keymaps`	399	108	250	2	0
`electric-pair`	375	122	87	110	16
`compile`	334	155	95	43	26
`theme`	206	96	46	34	30
`buffer-list`	198	104	47	18	29
`completion-at-point`	185	44	61	55	35
`occur`	180	89	46	15	26
`risp-mode`	143	34	68	19	32

Modulo	righe	codice	commenti	docstring	intestazione
`preview`	139	50	55	19	34
`rust-mode`	131	35	71	3	12
`debug`	118	40	43	22	0
`shell`	105	45	34	9	28
`cc-mode`	95	44	24	3	1
`commands`	80	9	20	39	0
`clipboard`	47	6	31	6	22
`minibuffer`	31	9	6	11	0

Righe; «intestazione» è il blocco `;;;` in testa al file, compreso nei commenti. Le intestazioni sono saggi di progetto: quella di `dired` spiega in 51 righe perché lo stato è il buffer stesso e perché il formato è così semplice. Le docstring delle funzioni private fanno spesso da commento: quella di `dired--parent` (992 caratteri) racconta come la funzione era scritta prima.

3.5 I manpage

Le 22 pagine sono lunghe 2 225 righe. Hanno tutte NAME e DESCRIPTION; 21 hanno SEE ALSO, 7 una sezione KEYS, 3 una SYNOPSIS, nessuna una sezione EXAMPLES. Gli esempi di codice sono concentrati in tre pagine (`writing-commands`, `configuration`, `background`); dieci pagine non ne hanno.

Misura	Valore
nomi pubblici: primitive e definizioni dei moduli	487, di cui 234 citati in almeno una pagina (48 %)
comandi	180, di cui 129 citati (72 %)
completamente coperti	`overlays`, `virtual_text`, `workers`
quasi scoperti	`completion.lisp` 1 su 11, `primitives/completion.rs` 1 su 8, `help.rs` 1 su 6, `mouse.rs` 1 su 6, `minibuffer.rs` 2 su 12, `results.rs` 2 su 9; nessuno di `completion-at-point`, `preview`, `clipboard`
impostazioni lette dal Rust	18: 4 citate in una pagina, 1 sola (`help-key`) con una docstring per `C-h v`
moduli senza una pagina che li descriva	`dired` (solo dentro `files`), `shell`, `indent`, `electric-pair`, `clipboard`, `preview`, `cc-mode`, `completion-at-point`
argomenti senza pagina	il log e la diagnostica, il mouse, l'indentazione

La copertura conta un nome come coperto se una pagina lo nomina anche solo di passaggio: le pagine che lo spiegano davvero sono meno.

3.6 I documenti in `doc/` e nella radice

File	righe	Stato	Destinazione
`architettura-editor.typ`	2 642	Buono fino al lavoro asincrono. Il capitolo «Il lavoro in background» descrive lo scheduler tolto (`BackgroundScheduler`, `RunNow`, `WorkerMessage`, `background/worker.rs`); il log è ancora «nella directory corrente»; ritmi di 40 ms che non esistono più; l'appendice delle	il manuale per sviluppatori: capitoli riscritti nella fase C

File	righe	Stato	Destinazione
		primitive è una copia generata e già vecchia.	
`workmanager.typ`	786	Documento di progetto: si dichiara «da implementare», mentre le patch 1–9 l'hanno implementato; il piano e le decisioni sono storia.	nel capitolo sul lavoro asincrono, poi si toglie
`lisp-fibers.typ`	870	In inglese. Descrive l'interno del crate `risp`: tutti i suoi percorsi e 16 dei suoi identificatori sono là.	il repository `risp` (decisione 5)
`rapporto-problemi.typ`	1 244	80 schede sulla revisione `08a853d`, nessuna segnata come risolta. Le 29 delle aree LET e INT riguardano l'interprete, ora in `risp`; rimanda ad `architettura-interprete.typ`, spostato in `risp` con `10598fa`. Almeno AVV-1, AVV-3, AVV-4 e BKG-1 sembrano risolte dalle patch.	aggiornato nel passo 10
`style.typ`	323	Gli aiuti condivisi per i diagrammi.	resta
`lisp-interpreter.pdf` `lisp-interpreter-flow.pdf`	—	PDF senza sorgente, dal 24 settembre.	da togliere (decisione 4)
`README.org`	—	Dice `Ctrl+S` per salvare e `Ctrl+Q` per uscire (sono `C-x C-s` e `C-x C-c`; `C-s` cerca); ripete metà della pagina `vim`; rimanda a `M-x rsedit-man`, che non esiste.	presentazione e compilazione, poi i rimandi
`PERFORMANCE.md`	—	Valido, salvo l'ultimo paragrafo, un'istruzione di una volta sola per applicare due patch.	resta, senza l'ultimo paragrafo
`man/README.md`	—	Dice che le pagine sono per argomenti e che la docstring «is the reference»: il contrario della regola nuova.	riscritto nel passo 1

3.7 Lo stesso fatto in più posti

Fatto	Dove	Come divergono
I moduli caricati all'avvio	`DEFAULT_INIT_LISP` (22 moduli), `build.rs` (22), `shipped_lisp_tests` (19), `doc_faithfulness_tests` (18), la sezione MODULES di `rsedit.txt` (19), `architettura-editor.typ` (21)	A `shipped_lisp_tests` mancano `cc-mode`, `preview` e `completion-at-point`; a `doc_faithfulness_tests` mancano `preview`, `theme`, `vim` e `completion-at-point`, le cui docstring quindi non sono controllate. La pagina non ha `cc-mode`, `preview` e `vim`; il manuale non ha `vim`.
L'elenco delle pagine	la directory `man/` (22), l'elenco di `guide_tests` (21), l'indice di `guide.txt`	Manca `vim`: la guida non la elenca, `rsedit.txt` non la cita e i suoi tasti non sono controllati.

Fatto	Dove	Come divergono
Dove sta la configurazione	`paths.rs`, `configuration.txt`, `architettura-editor.typ`	Pagina e manuale non dicono nulla della cache e del log; il manuale ha il log nella directory corrente.
Le impostazioni	le costanti dei moduli Rust, la tabella del manuale, i commenti di `DEFAULT_INIT_LISP` e di `common-keymaps.lisp`	`configuration.txt` dice che `C-h v` spiega ogni variabile: vale per 1 impostazione su 18.
L'ordine dei moduli	`DEFAULT_INIT_LISP`, `configuration.txt`	«The order matters» contro «None of them needs any other».
Le primitive	il registro (`into.function`, `into.command`), l'appendice del manuale	L'appendice ne dà 4 a `workers` (sono 9) e 47 a `edits` (sono 48).
`background-call` e i lavori Lisp	la docstring, `background.txt`, `workmanager.typ`, il manuale, le 78 righe di `//!` di `primitives/lisp_job.rs`	Stesso esempio e stesse regole in tre posti; il manuale descrive lo scheduler di prima.
La modalità vim	`vim.txt`, `README.org`	Il README ripete comandi e impostazioni.

3.8 Nomi citati che non esistono

- `rsedit-man`, in cinque posti: `man/README.md`, `README.org`, la docstring di `data-directory`, un commento di `build.rs` e `vim.lisp`. Il comando è `manpage`.
- Nelle docstring: `read-command-args` (in `primitives/commands.rs`), `search-forward` (in quella di `scan-buffer`), `find-file-recursive` (il comando è `dired-find-recursive`), `default-directory` (in quella di `grep`). Chi le legge con `C-h f` e segue il nome non trova nulla.
- Nei commenti: `set-mode-keywords` (`rust-mode.lisp`); nomi di Emacs al posto di quelli di rsedit, come `self-insert-command`.
- Nei documenti typst: 8 identificatori del manuale e 5 di `workmanager.typ` non esistono più, come il `background/mod.rs` citato dal rapporto; 16 percorsi del rapporto e tutti quelli di `lisp-fibers.typ` sono ora nel crate `risp`.

3.9 Difetti trovati durante l'inventario

Non sono di documentazione, ma l'inventario li ha fatti emergere.

1. **`:help` di vim non fa nulla.** `vim.lisp` chiama `rsedit-man` solo se esiste, e non esiste: il comando resta muto.
2. **La pagina `vim` non si raggiunge dalla guida** e i suoi tasti non sono controllati (§ 3.7).
3. **I moduli `theme`, `vim`, `preview` e `completion-at-point` sfuggono a `doc_faithfulness_tests`**, che carica un elenco suo di moduli.
4. **`configuration.txt` consiglia `(eval-file "clipboard")`**, che l'`init.lisp` predefinito carica già.

4 Il piano

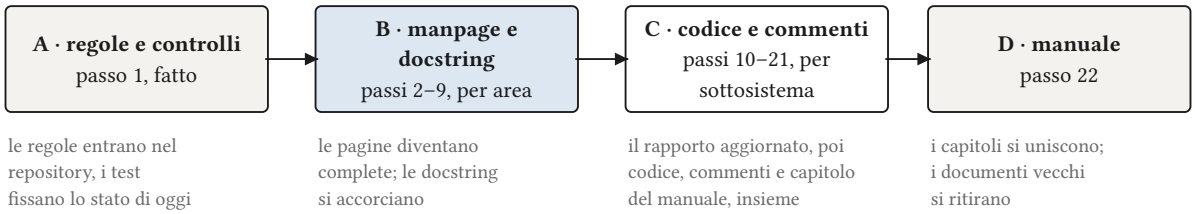


Figura 2: Le quattro fasi. Ogni passo è una patch, consegnata e verificata come le patch 1–9.

L'ordine è quello concordato: prima i manpage insieme alle docstring, poi il codice e i commenti modulo per modulo, infine il manuale. Con una precisazione: i perché tolti dal codice nella fase C vanno subito nel capitolo del manuale del loro sottosistema, così nessuna informazione resta senza casa tra un passo e l'altro. La fase D unisce i capitoli, disegna i diagrammi d'insieme e ritira i documenti vecchi (decisione 9).

4.1 Fase A — le regole e i controlli (passo 1, fatto)

1. `README.md` prende il posto di `README.org`: una presentazione essenziale del progetto, con i documenti e i loro destinatari. `man/README.md` descrive il modello di pagina del § 2.3.
2. **Un solo elenco dei moduli.** `build.rs` copia tutta `core/lisp/`; i test leggono l'elenco da `DEFAULT_INIT_LISP`; `guide_tests` confronta con quello la sezione MODULES di `rsedit.txt`, che ora ha tutti i 22 moduli.
3. **Un solo elenco delle pagine.** I test leggono la directory `man/`.
4. **I test nuovi**, con le eccezioni di oggi in `core/src/tests/doc_exceptions/`: 459 nomi senza casa, 30 docstring oltre i 1 000 caratteri, 68 funzioni private di `vim.lisp` senza docstring, 11 nomi citati che l'editor non definisce (5 variabili lette dal Rust e mai dichiarate, 6 nomi sbagliati).
5. **Il rimando automatico.** `C-h f` e `C-h v` mostrano «Manual: pagina», e `K` o `RET` su quella riga apre la pagina.
6. **Le correzioni del § 3.9.** `:help` di vim apre la pagina `vim`, che ora è nella guida, in `rsedit.txt` e nei test; tutti i moduli sono controllati. Le SYNOPSIS di `compile`, `themes` e `vim` elencano nomi, e i loro tasti sono passati in sezioni KEYS controllate.
7. Tolti i due PDF orfani e l'ultimo paragrafo di `PERFORMANCE.md`.

4.2 Fase B — i manpage e le docstring (passi 2–9)

Ogni passo copre un'area. Le pagine dell'area si riscrivono sul modello, con SYNOPSIS, EXAMPLES e KEYS; le pagine mancanti si scrivono. Le docstring dei nomi dell'area si accorciano sul modello. Le spiegazioni per l'utente che stanno nei commenti del codice dell'area passano nelle pagine e dai commenti escono. Le impostazioni dell'area ricevono una docstring, così che `C-h v` le spieghi. L'elenco dei nomi senza casa si accorcia di conseguenza.

Passo	Area	Pagine	Primitive (`primitives/`)	Moduli Lisp e pagine nuove
2	avvio, configurazione, diagnostica	`rsedit`, `guide`, `first-steps`, `configuration`	da `general.rs`: `eval-file`, `log`, `all-logs`, `backtrace`	`debug`, `commands`; nuova: `diagnostics` (il log nella cache, `*Messages*`, `C-h e`, il backtrace, le righe del watchdog)
3	scrivere e modificare	`editing`, `rectangles`, `macros`	`edits`, `verbs`, `region`, `rectangle`,	`indent`, `electric-pair`, `clipboard`; nuova: `indentation`

Passo	Area	Pagine	Primitive (`primitives/`)	Moduli Lisp e pagine nuove
			`macros`, `comments`	
4	file, buffer, finestre	`files`, `buffers`, `windows`	`io`, `buffers`, `windows`, `ui`, `mouse`, `ask`	`dired`, `buffer-list`; nuove: `dired`, `mouse`; le impostazioni `auto-save` e `watch-files`
5	cercare e navigare	`searching`, `compile`	`isearch`, `replace`, `results`, `scan`, `match_data`	`occur`, `compile`
6	prompt, completamento, tasti, aiuto	`completion`, `keymaps`, `getting-help`	`minibuffer`, `completion`, `help`, `commands`, il resto di `general`	`minibuffer`, `completion`, `completion-at-point`, `help`, `manpage`, `preview`, `common-keymaps`
7	modi e aspetto	`modes`, `themes`, `overlays`, `virtual-text`	`modes`, `theme`, `overlays`, `virtual_text`	`rust-mode`, `risp-mode`, `cc-mode`, `theme`
8	estendere l'editor	`writing-commands` `background`	`workers`, `shell`	`shell`; nuova: `shell`
9	vim	`vim`	—	`vim`

4.3 Fase C — il codice e i commenti (passi 10–21)

Il passo 10 aggiorna il rapporto sui problemi: ogni scheda verificata sul codice di quel momento, il campo `stato`, le schede dell'interprete separate per il repository `risp`. Va prima del codice perché la classificazione dei commenti (§ 2.13) deve sapere quali problemi sono risolti.

Gli altri passi seguono l'ordine delle dipendenze di `lib.rs`, dal basso. Ogni passo comincia con una proposta breve degli spostamenti — file da dividere, unire o rinominare — che approvi prima della patch. Poi la patch: il rustdoc del sottosistema secondo le regole, i commenti classificati, il capitolo del manuale con le sue decisioni e i suoi diagrammi, i commenti dei test del sottosistema (decisione 8). Il comportamento non cambia: tutti i test passano senza modifiche, salvo gli spostamenti che seguono quelli del codice.

Passo	Parte	Righe	Note
10	`doc/rapporto-problemi.typ`	1 244	80 schede da verificare
11	`work/`	2 935	assorbe `workmanager.typ`
12	`paths.rs`, `buffer/`	3 075	—
13	`text/`, `input.rs`, `commands.rs`	2 047	—
14	`managers/`	2 936	la regola delle coppie con `editor/`
15	`editor/`	4 838	`boot.rs`, `keys.rs`, `frame.rs`
16	`primitives/`, modifica del testo: `edits`, `verbs`, `region`, `rectangle`, `macros`, `comments`, `match_data`, `scan`	4 353	docstring già fatte nella fase B

Passo	Parte	Righe	Note
17	`primitives/`, file, finestre e lavori: `io`, `buffers`, `windows`, `ui`, `mouse`, `ask`, `shell`, `workers`, `lisp_job`, `args`	4 995	`io.rs` è il file più lungo: 1 818 righe
18	`primitives/`, il resto: `commands`, `completion`, `general`, `help`, `isearch`, `minibuffer`, `modes`, `overlays`, `replace`, `results`, `theme`, `virtual_text`, `mod`	5 953	—
19	`modes/`, `feature/`	2 896	—
20	`ui/`, `src/` (la TUI)	3 408	—
21	`core/lisp/`: intestazioni e commenti	6 422	docstring già fatte nella fase B

I criteri con cui si propongono gli spostamenti:

- **La mappa di `lib.rs` è quella vera.** Oggi elenca un modulo `lisp/` che è nel crate `risp`. Un test la confronta con le dichiarazioni `mod`.
- **Un concetto per file, e il file ha il nome del concetto.** Un file oltre le 800 righe circa deve avere un motivo: oggi `primitives/io.rs` (1 818), `primitives/edits.rs` (1 567), `ui/windows.rs` (1 437), `work/manager.rs` (1 094), `text/search.rs` (952), `primitives/results.rs` (936), `src/tui.rs` (924).
- **Una regola sola per le coppie `editor/X` e `managers/X`**: ce ne sono otto (`buffers`, `commands`, `kill_yank`, `log`, `macros`, `modes`, `runtime`, `windows`). La regola si scrive in `lib.rs` e nel manuale, e vale per tutte.
- **La stessa parola per la stessa cosa**: lavoro, worker e task, per esempio.
- **In un file, prima ciò che è pubblico**, poi gli aiuti privati, nell'ordine in cui il lettore li incontra.
- **Un file di test per argomento**, con il nome del modulo che prova.

4.4 Fase D — il manuale (passo 22)

- `architettura-editor.typ` diventa il manuale per sviluppatori: l'introduzione, i diagrammi d'insieme (strati, thread, ordine dei lock), i capitoli scritti nella fase C, le ricette, il glossario.
- `workmanager.typ` si toglie, assorbito nel passo 11.
- L'appendice generata delle primitive si toglie: è una copia del registro, e `C-h a` la sostituisce.
- Da questo documento escono l'inventario e il piano; restano le regole.
- I PDF si rigenerano.

4.5 Come si consegna ogni passo

Come le patch 1–9. La patch si prepara su una copia di lavoro, si prova con `git am` su una copia pulita, con l'intera suite di test, e confrontando gli avvisi di clippy e di `cargo doc` con quelli della base. Arriva con il PDF dei documenti toccati e con la spiegazione, file per file, di che cosa cambia e di dove differisce da questo piano.

5 Le decisioni

Le risposte alle domande del passo 0.

1. **Il README.** `man/README.md` si riscrive con il modello delle pagine. Il README del progetto diventa una descrizione essenziale secondo le pratiche di GitHub, un portfolio sobrio dell'applicazione, con l'elenco dei documenti e dei loro destinatari: `README.md` al posto di `README.org`.
2. **La SYNOPSIS come casa dei nomi**, con il rimando automatico in `C-h f`: sì.
3. **Un solo elenco dei moduli**: `build.rs` copia tutta la directory.
4. **I PDF in git**: restano, rigenerati nello stesso commit della sorgente; i due orfani si tolgono.
5. **`lisp-fibers.typ`**: lo toglie l'autore.
6. **`workmanager.typ`**: ciò che è ancora valido confluisce nel manuale nel passo 11, e il documento si toglie nella fase D.
7. **La storia nei commit**: le patch si applicano con `git am`.
8. **I commenti dei test**: si ripuliscono con le stesse regole insieme al loro sottosistema, nella fase C.
9. **Il manuale scritto durante la fase C**: sì.
10. **Le correzioni dell'inventario**: quelle di codice e test nel passo 1, quella di `configuration.txt` nel passo 2.